COLLEGE ENGINEERING PROJECT · PROJECT 27 OF 55

Retrieval-Augmented AI System for Intelligent Question Answering from Academic Documents

Domain: Retrieval-Augmented Generation & EdTech | **Module:** project_27_academic_qa_rag

PROJECT ABSTRACT

Academic institutions and students work with a large volume of digital resources such as textbooks, lecture notes, research papers, question banks, laboratory manuals, and project documents. Finding specific information from lengthy academic documents can be time-consuming, while conventional keyword-based search systems often fail to understand the actual meaning or context of a user's question. The proposed Retrieval-Augmented AI System for Intelligent Question Answering from Academic Documents aims to provide an intelligent solution that allows users to ask questions in natural language and receive answers based specifically on the information available in their academic documents. The proposed system is implemented using Python, Natural Language Processing, text embeddings, vector database technology, and Large Language Models (LLMs). Uploaded documents are first processed, cleaned, and divided into meaningful text segments. These segments are converted into numerical vector representations using embedding techniques and stored in a vector database. When a user asks a question, the system searches the stored document representations and retrieves the most semantically relevant content before generating the final answer. The major uniqueness of the proposed system is its retrieval-first approach. Unlike a general AI chatbot that generates responses mainly from its pre-trained knowledge, this system grounds its responses in the user's selected academic documents. This allows the system to provide answers that are more relevant to the actual learning material. The semantic retrieval mechanism can also identify related information even when the words used in the user's question are different from the exact wording present in the document. Another important feature is source-aware answer generation. The system can identify the document or relevant section from which the answer was derived, allowing students to verify the generated response against the original academic material. This improves transparency and reduces the risk of users accepting an AI-generated answer without understanding its source. The system can also support follow-up questions while maintaining the context of the ongoing academic discussion. The proposed system can be useful for students, faculty members, educational institutions, and academic researchers. Students can use it to clarify doubts, locate information from lengthy study materials, and support examination preparation, while faculty members can use it to quickly access information from subject-specific documents. New documents can also be added to the system without retraining the underlying language model, making the solution flexible and scalable for different subjects and academic environments. Overall, the project demonstrates how Retrieval-Augmented Generation (RAG) can transform static academic documents into an interactive and context-aware knowledge system. By combining semantic document retrieval,

> vector-based search, and AI-generated responses with source references, the proposed system provides a practical approach to intelligent academic question answering while keeping responses closely connected to the actual documents provided by the user.

📌 Project Metadata & Specifications

Project Title	Retrieval-Augmented AI System for Intelligent Question Answering from Academic Documents
Domain / Specialization	Retrieval-Augmented Generation & EdTech
Source Directory	`project_27_academic_qa_rag`
Backend REST API	`http://localhost:5027` (Flask Python)
Frontend Interface	Interactive Dashboard (`frontend/index.html`)
Execution Mode	Hybrid Dual-Mode (Online API & Offline Autonomous Simulation)

🏗️ System Methodology & Implementation Phases

PROJECT 27 — PHASE 1

Phase 1 — Requirements, Data and Prototype

The first phase will convert the project concept into a measurable prototype for NLP/language or intelligent information processing. The work begins by defining the exact input, output, user flow and evaluation criteria from the project abstract. A small requirements specification will identify the primary prediction/detection task, expected classes or target value, acceptable response time, and the form in which results must be displayed.

1. Problem Definition and Scope

Define the operational scenario, target users, system boundaries, assumptions, success criteria and non-functional requirements. Separate what the model must predict from what the application must display or recommend. Identify cases that should be rejected, flagged for review, or returned with low confidence.

2. Data Preparation

Identify a suitable dataset or approved collection procedure matching the abstract. Build a data dictionary, remove unusable records, handle missing values, normalize or standardize numerical fields where appropriate, and verify labels. For image/video/audio/text projects, define preprocessing, augmentation and quality-control rules. Split data into training, validation and test sets without leakage.

3. Baseline Model and Experiment Setup

Implement a reproducible baseline using the algorithm/model family stated in the abstract when one is specified. Establish a simple reference result before introducing additional complexity. Record dataset version, preprocessing settings, model configuration, random seed, training time and evaluation metrics so that later improvements can be compared objectively.

4. Initial Application Prototype

Build the minimum user-facing workflow needed to demonstrate the core task. The prototype should accept valid input, run preprocessing, invoke the baseline model, and return the predicted result with an understandable confidence/score where supported. Keep model logic separate from the interface so the model can be replaced without rewriting the application.

Phase 1 Deliverables

Requirements and scope note; prepared dataset or data pipeline; baseline model; experiment log; initial application/prototype; and a short Phase 1 validation report documenting what works, what remains uncertain, and the measurements to be improved in Phase 2.

Project-Specific Phase 1 Work Items

- Create a labelled visual dataset structure and define image/video quality checks.
- Set a consistent image resize/normalization pipeline and establish augmentation rules.
- Record sample predictions visually so incorrect detections can be traced back to input regions.

PROJECT 27 — PHASE 2

Phase 2 — Optimization, Integration and Validation

The second phase will turn the Phase 1 prototype into a validated project for NLP/language or intelligent information processing. The emphasis is on improving model reliability, connecting the complete workflow, testing realistic edge cases, and producing evidence that the final system meets the defined objectives.

1. Model Improvement

Compare relevant model configurations, tune important hyperparameters, address class imbalance where present, and evaluate whether additional features or augmentation improve generalization. Use validation data for tuning and reserve the test set for final evaluation. Prefer measurable improvements over unnecessary model complexity.

2. End-to-End Integration

Connect the data/input layer, preprocessing, inference engine, result-processing logic, database or file storage where required, and user interface. Add input validation, error handling, logging and clear status messages. If recommendations, alerts, summaries, risk scores or dashboards are part of the abstract, implement them as a separate decision-support layer around the model.

3. Testing and Evaluation

Perform functional, model, integration and usability tests. Evaluate accuracy or regression error as appropriate, along with precision, recall, F1, confusion matrix, ROC-AUC, latency or other task-specific measures where relevant. Test difficult, noisy, incomplete and out-of-distribution inputs and document failure cases instead of hiding them.

4. Demonstration and Final Evidence

Prepare representative test cases, screenshots or result tables, model-comparison evidence and a final demonstration workflow. Document limitations, data constraints, privacy/security considerations where applicable, and future extensions. The final report should distinguish measured results from proposed future work.

Phase 2 Deliverables

Optimized model; integrated application; test and evaluation report; representative evidence; limitations and risk register; deployment/demo instructions; and final project documentation suitable for academic review or demonstration.

Project-Specific Phase 2 Validation

- Compare the final model with the Phase 1 baseline using the same held-out evaluation set.
- Document representative success cases and failure cases instead of reporting only aggregate accuracy.
- Prepare a final demonstration script covering input, processing, output, validation and limitations.

Final Expected Outcome
A reproducible, tested prototype that implements the project concept described in the supplied abstract, together with measurable evaluation evidence and clear documentation of limitations, assumptions and future enhancement opportunities.

🚀 Local Deployment & Execution Guide

Follow the steps below to initialize and demonstrate the system:

1. Backend Server Setup

```
# Navigate to the backend directory cd "/Users/balamurugan/Documents/orbyte globel /college project/project_27_academic_qa_rag/backend" # Install project dependencies pip install -r requirements.txt # Run the Flask API service python3 app.py
```

The backend microservice will listen for requests at `http://localhost:5027`.

2. Frontend User Interface

- Open `/Users/balamurugan/Documents/orbyte globel /college project/project_27_academic_qa_rag/frontend/index.html` directly in Google Chrome, Firefox, or Safari.
- Alternatively, launch via the central **Master Launchpad Portal (`index.html`)** located at the root directory.
- The user interface is designed with **hybrid dual-mode resilience**: when the backend API is live, real-time inferences are queried; when offline, authentic client-side mathematical models simulate sensor telematics.